Memory aware SCADE

Jean-Louis Colaço, **Baptiste Pauget**, Marc Pouzet

Synchron 2022

Dec 2022, the 1st

Objectives

Drive memory use in a purely functional (data-flow) language

Objectives

Drive memory use in a purely functional (data-flow) language

Reuse memory: arrays, structures

- Limit memory footprint
- Avoid copies (in-place modification)

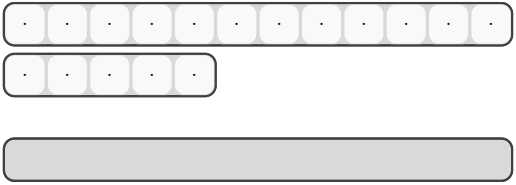
Objectives

Drive memory use in a purely functional (data-flow) language

Reuse memory: arrays, structures

- Limit memory footprint
- Avoid copies (in-place modification)

Example Padded convolution



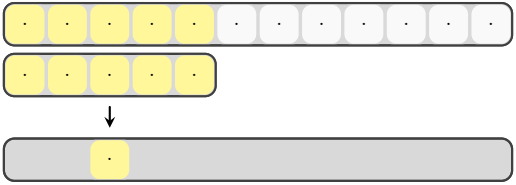
Objectives

Drive memory use in a purely functional (data-flow) language

Reuse memory: arrays, structures

- Limit memory footprint
- Avoid copies (in-place modification)

Example Padded convolution



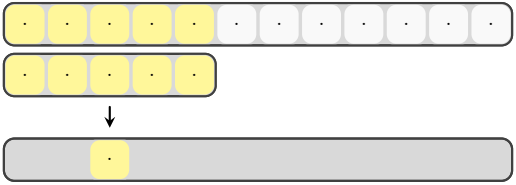
Objectives

Drive memory use in a purely functional (data-flow) language

Reuse memory: arrays, structures

- Limit memory footprint
- Avoid copies (in-place modification)

Example Padded convolution



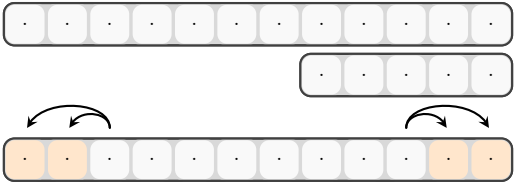
Objectives

Drive memory use in a purely functional (data-flow) language

Reuse memory: arrays, structures

- Limit memory footprint
- Avoid copies (in-place modification)

Example Padded convolution



Objectives

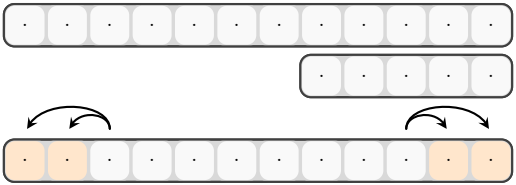
Drive memory use in a purely functional (data-flow) language

Reuse memory: arrays, structures

- Limit memory footprint
- Avoid copies (in-place modification)

e.g. Built-in-place optimization of Sisal[1]

Example Padded convolution



[1] J-L Gaudiot et al. "The Sisal model of functional programming and its implementation". In: *Proceedings of IEEE International Symposium on Parallel Algorithms Architecture Synthesis*. IEEE. 1997, pp. 112–123.

Objectives

Drive memory use in a purely functional (data-flow) language

Reuse memory: arrays, structures

- Limit memory footprint
- Avoid copies (in-place modification)

e.g. Built-in-place optimization of Sisal[1]

Calling imperative code: safely encapsulate argument modification

[1] J-L Gaudiot et al. "The Sisal model of functional programming and its implementation". In: *Proceedings of IEEE International Symposium on Parallel Algorithms Architecture Synthesis*. IEEE. 1997, pp. 112–123.

Objectives

Drive memory use in a purely functional (data-flow) language

Reuse memory: arrays, structures

- Limit memory footprint
- Avoid copies (in-place modification)

e.g. Built-in-place optimization of Sisal[1]

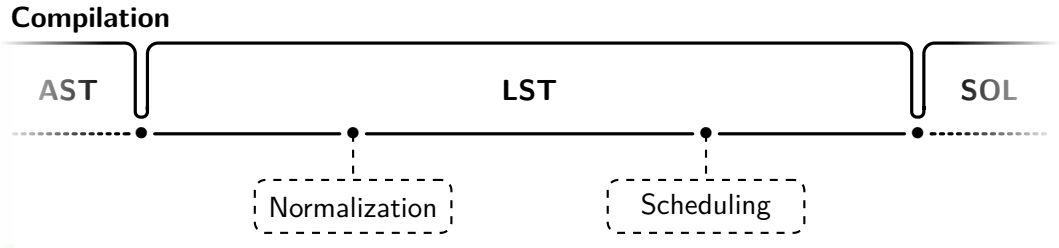
Calling imperative code: safely encapsulate argument modification

... while preserving

- the data-flow style (fully parallel)
- referential transparency

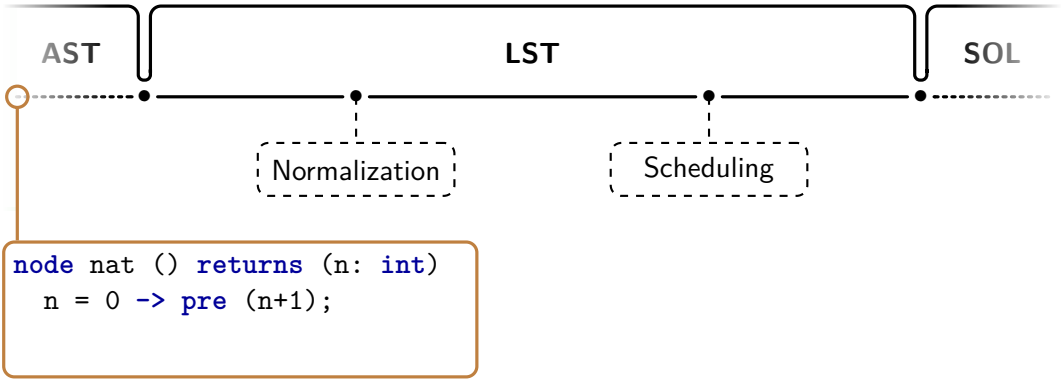
[1] J-L Gaudiot et al. "The Sisal model of functional programming and its implementation". In: *Proceedings of IEEE International Symposium on Parallel Algorithms Architecture Synthesis*. IEEE. 1997, pp. 112–123.

Compiler middle-end



Compiler middle-end

Compilation

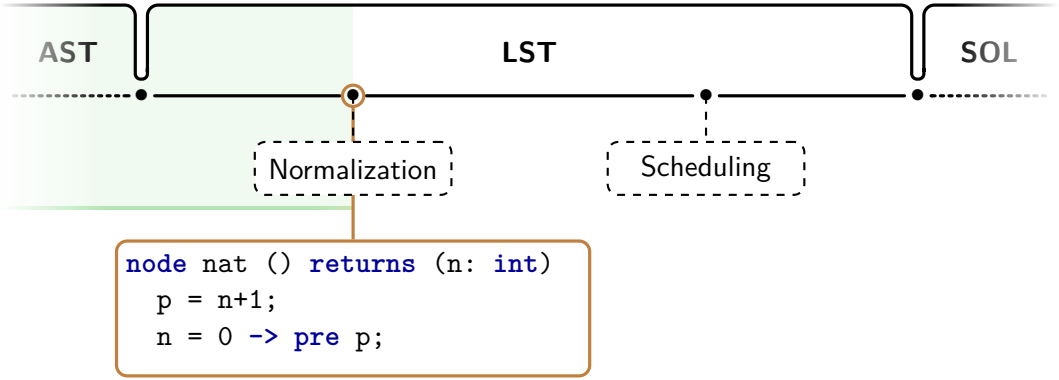


- Referential transparency
- Data-flow representation

⇒ **Implicit copies**

Compiler middle-end

Compilation

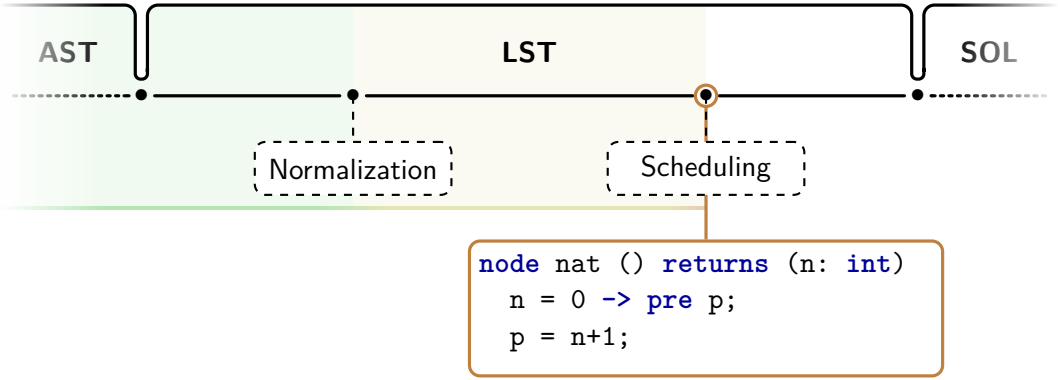


- ~~Referential transparency~~
- Data-flow representation

⇒ **Identified memory (state)**

Compiler middle-end

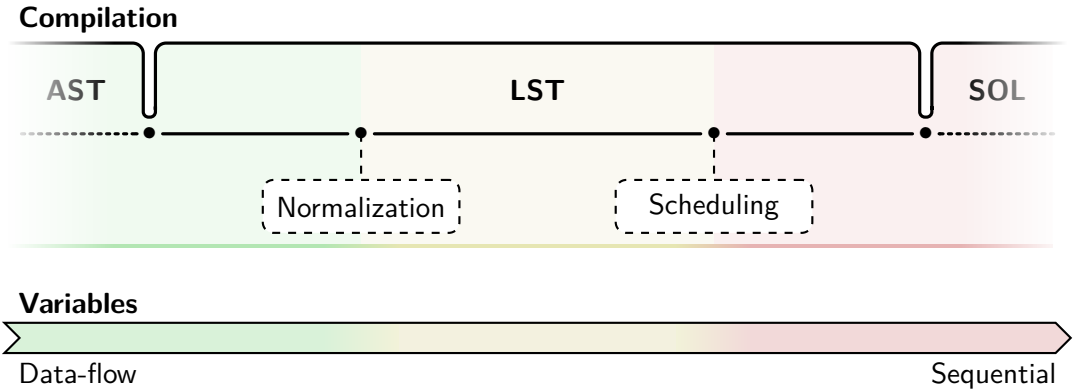
Compilation



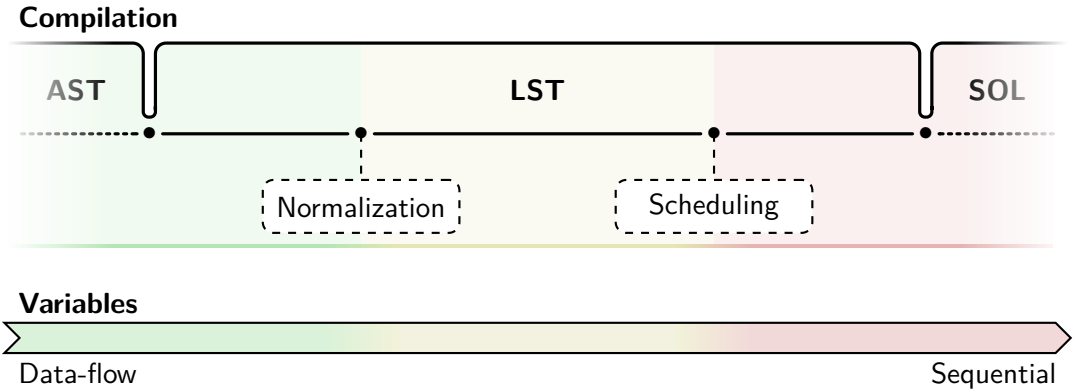
- ~~Referential transparency~~
- ~~Data-flow representation~~

⇒ **Explicit copies**

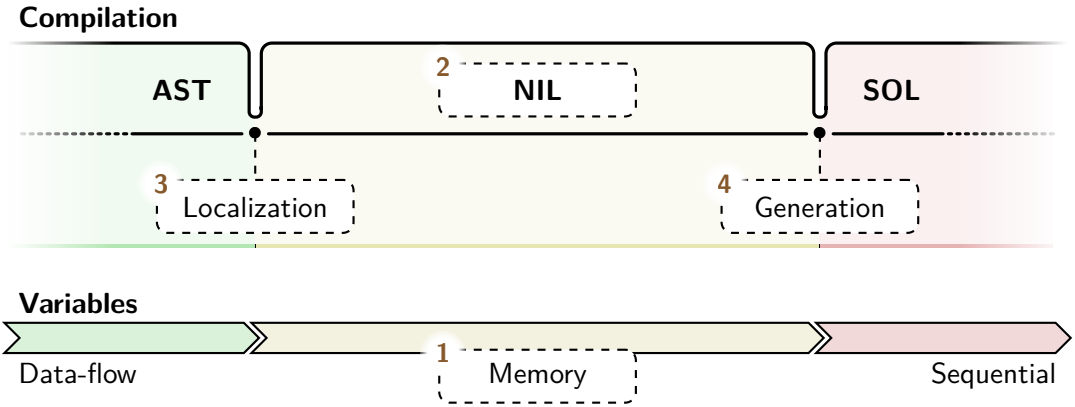
Compiler middle-end



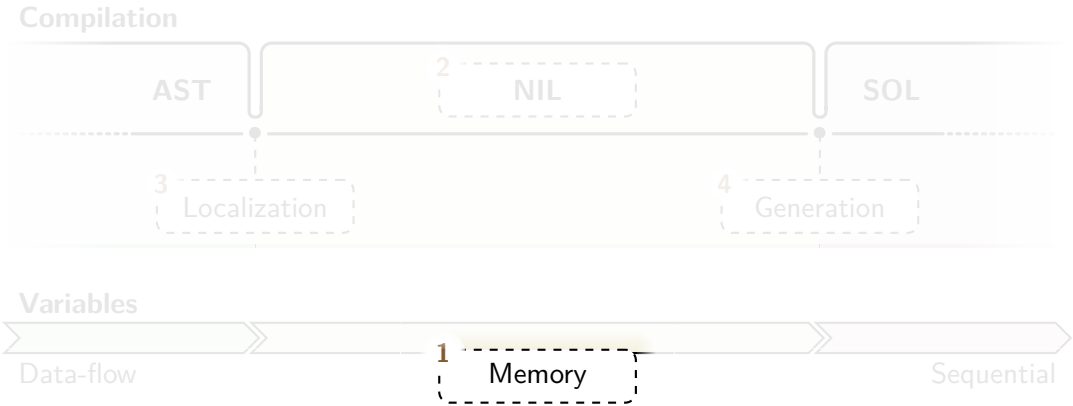
Compiler middle-end



Agenda


Compilation
AST
2
NIL
SOL
3
Localization
4
Generation
Variables
Data-flow
1
Memory
Sequential

Agenda


Compilation
AST
2
NIL
SOL
3
Localization
4
Generation
Variables
Data-flow
1
Memory
Sequential

What's in memory ?

Structured data

τ	$::=$		***Types***
	$\mid$	$\alpha,\ \beta,\ \gamma$	variable
	$\mid$	$\texttt{int} \mid \texttt{bool}$	base types
	$\mid$	$\tau * \ldots * \tau$	tuple
	$\mid$	$[n]\tau$	array

What's in memory ?

Structured data

τ	$::=$		***Types***
	$\mid$	$\alpha,\ \beta,\ \gamma$	variable
	$\mid$	$\texttt{int} \mid \texttt{bool}$	base types
	$\mid$	$\tau * \ldots * \tau$	tuple
	$\mid$	$[n]\tau$	array

Memory location

$$\varepsilon.f$$

Typed memory variable

Projection function

Projection functions

Intuition: Viewing (part of) structured data differently by mapping accesses

Projection functions

Intuition: Viewing (part of) structured data differently by mapping accesses

Array transposition:

$$[n]\,[p]\,\tau \Longleftarrow \Big[\texttt{[j<}n\texttt{]}\ \texttt{[i<}p\texttt{]} \leftarrow \texttt{[i<}p\texttt{]}\ \texttt{[j<}n\texttt{]}\Big] \Longrightarrow [p]\,[n]\,\tau$$

Projection functions

Intuition: Viewing (part of) structured data differently by mapping accesses

Array transposition:

$$[n]\,[p]\,\tau \Longrightarrow \left[\texttt{[j<}n\texttt{]}\ \texttt{[i<}p\texttt{]}\ \leftarrow\ \texttt{[i<}p\texttt{]}\ \texttt{[j<}n\texttt{]}\right] \Longrightarrow [p]\,[n]\,\tau$$

Array splitting:

$$[n+p]\,\tau \Longrightarrow \begin{bmatrix} \texttt{[i+0<}n+p\texttt{]} & \leftarrow & \texttt{.1}/2\ \texttt{[i<}n\texttt{]} \\ \texttt{[i+n<}n+p\texttt{]} & \leftarrow & \texttt{.2}/2\ \texttt{[i<}p\texttt{]} \end{bmatrix} \Longrightarrow [n]\,\tau * [p]\,\tau$$

Operations on projection functions

Properties

- Injective $\Longrightarrow$ ✓ Editing in place

slices

$$[n+p-1]\,\alpha = \left[\texttt{[i+j<}n+p-1\texttt{]} \;\leftarrow\; \texttt{[i<}n\texttt{]}\;\texttt{[j<}p\texttt{]}\right] \Longrightarrow [n]\,[p]\,\alpha$$

Operations on projection functions

Properties

- Injective $\Longrightarrow$ ✓ Editing in place
- Single piece $\Longrightarrow$ ✓ Passing by pointer

---

map snd

$$[n](\alpha * \beta) = \left[\texttt{[i<}n\texttt{]}\ \texttt{.2}/2 \leftarrow \texttt{[i<}n\texttt{]}\right] \Longrightarrow [n]\beta$$

Operations on projection functions

Properties

- Injective $\Longrightarrow$ ✓ Editing in place
- Single piece $\Longrightarrow$ ✓ Passing by pointer

Comparison

- Inclusion
- Disjunction

$\Longrightarrow$ Dependencies: $\cap\, p_i \setminus \cup q_i$

Operations on projection functions

Properties

- Injective $\Longrightarrow$ ✓ Editing in place
- Single piece $\Longrightarrow$ ✓ Passing by pointer

Comparison

- Inclusion
- Disjunction

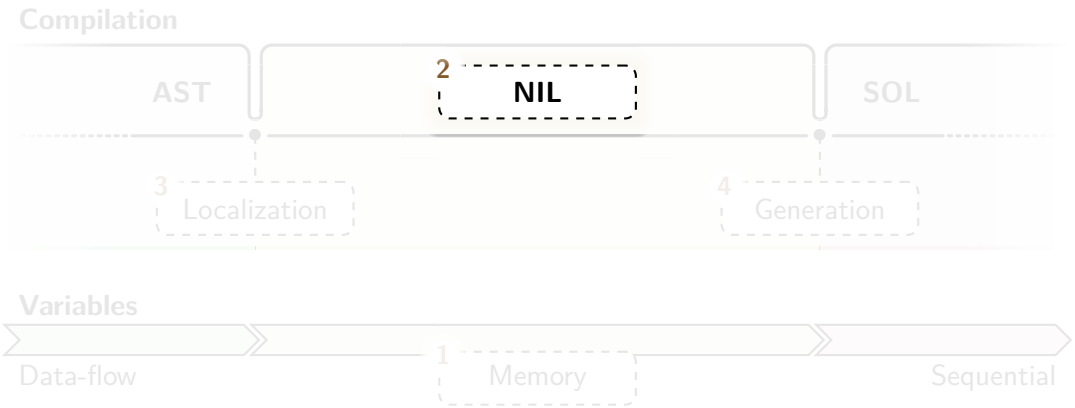
$\Longrightarrow$ Dependencies: $\cap\, p_i \setminus \cup q_i$

Transformations

- Composition
- Normal form (simplification)

$\Longrightarrow$ $\mathtt{tr}_{[n][p]\alpha} \circ \mathtt{tr}_{[p][n]\alpha} = I_{[n][p]\alpha}$

Agenda


Compilation
AST
2
NIL
SOL
3
Localization
4
Generation
Variables
Data-flow
1
Memory
Sequential

No-copy Intermediate Language

No-copy Intermediate Language

e	$::=$		***Expressions***
	$\mid$	x	variable
	$\mid$	n	constant
	$\mid$	$e + e \mid \ldots$	operators
	$\mid$	$x\ (e, \ldots, e)$	instanciation
	$\mid$	e `->` e	initializer
	$\mid$	`pre` e	register

No-copy Intermediate Language

e	$::=$		***Expressions***
	$\mid$	x	variable
	$\mid$	n	constant
	$\mid$	$e + e \mid \ldots$	operators
	$\mid$	$x\ (e, \ldots, e)$	instanciation
	$\mid$	e `->` e	initializer
	$\mid$	`pre` e	register
	$\mid$	`!`e	copy

No-copy Intermediate Language

e	$::=$		***Expressions***
	$\mid$	x	variable
	$\mid$	n	constant
	$\mid$	$e + e \mid \ldots$	operators
	$\mid$	$x\ (e, \ldots, e)$	instanciation
	$\mid$	e `->` e	initializer
	$\mid$	`pre` e	register
	$\mid$	`!`e	copy
	$\mid$	$d(e)$	descriptor

$d(a)$	$::=$		***Descriptors***
	$\mid$	$(a, \ldots, a)$	tuple
	$\mid$	$[a, \ldots, a]$	array

No-copy Intermediate Language

e	$::=$		***Expressions***
	$\mid$	x	variable
	$\mid$	n	constant
	$\mid$	$e + e \mid \ldots$	operators
	$\mid$	$x\ (e, \ldots, e)$	instanciation
	$\mid$	e `->` e	initializer
	$\mid$	`pre` e	register
	$\mid$	`!`e	copy
	$\mid$	$d(e)$	descriptor

$d(a)$	$::=$		***Descriptors***
	$\mid$	$(a, \ldots, a)$	tuple
	$\mid$	$[a, \ldots, a]$	array
	$\mid$	a `++` a	concat

No-copy Intermediate Language

e	$::=$		***Expressions***
	$\mid$	x	variable
	$\mid$	n	constant
	$\mid$	$e + e \mid \ldots$	operators
	$\mid$	$x\ (e, \ldots, e)$	instanciation
	$\mid$	e `->` e	initializer
	$\mid$	`pre` e	register
	$\mid$	`!`e	copy
	$\mid$	$d(e)$	descriptor

$d(a)$	$::=$		***Descriptors***
	$\mid$	$(a, \ldots, a)$	tuple
	$\mid$	$[a, \ldots, a]$	array
	$\mid$	a `++` a	concat
	$\mid$	a `at` x	localization
	$\mid$	a `:` τ	typing

No-copy Intermediate Language

e	$::=$		***Expressions***
	$\mid$	x	variable
	$\mid$	n	constant
	$\mid$	$e + e \mid \ldots$	operators
	$\mid$	$x\ (e, \ldots, e)$	instanciation
	$\mid$	e `->` e	initializer
	$\mid$	`pre` e	register
	$\mid$	`!`e	copy
	$\mid$	$d(e)$	descriptor
eq	$::=$	$p = e;$	***Equations***

$d(a)$	$::=$		***Descriptors***
	$\mid$	$(a, \ldots, a)$	tuple
	$\mid$	$[a, \ldots, a]$	array
	$\mid$	a `++` a	concat
	$\mid$	a `at` x	localization
	$\mid$	a `:` τ	typing
p	$::=$		***Patterns***
	$\mid$	x	variable
	$\mid$	p `as` x	naming
	$\mid$	$d(p)$	descriptor

No-copy Intermediate Language

```
let swap (p) = (q at p):
  (a,b) = p;
  q = (!b, !!a);
```

e	$::=$		***Expressions***
	$\mid$	x	variable
	$\mid$	n	constant
	$\mid$	$e + e \mid \ldots$	operators
	$\mid$	$x\ (e, \ldots, e)$	instanciation
	$\mid$	e `->` e	initializer
	$\mid$	`pre` e	register
	$\mid$	`!`e	copy
	$\mid$	$d(e)$	descriptor
eq	$::=$	$p = e;$	***Equations***

$d(a)$	$::=$		***Descriptors***
	$\mid$	$(a, \ldots, a)$	tuple
	$\mid$	$[a, \ldots, a]$	array
	$\mid$	a `++` a	concat
	$\mid$	a `at` x	localization
	$\mid$	a `:` τ	typing
p	$::=$		***Patterns***
	$\mid$	x	variable
	$\mid$	p `as` x	naming
	$\mid$	$d(p)$	descriptor

No-copy Intermediate Language

```
let swap (p) = (q at p):
  (a,b) = p;
  q = (!b, !!a);
```

```
let set_right (a, x) = (b at a):
  l ++ _ = a;
  b = l ++ !x;
```

e	$::=$		***Expressions***
	$\mid$	x	variable
	$\mid$	n	constant
	$\mid$	$e + e \mid \ldots$	operators
	$\mid$	$x\ (e, \ldots, e)$	instanciation
	$\mid$	e `->` e	initializer
	$\mid$	`pre` e	register
	$\mid$	`!`e	copy
	$\mid$	$d(e)$	descriptor
eq	$::=$	$p = e;$	***Equations***

$d(a)$	$::=$		***Descriptors***
	$\mid$	$(a, \ldots, a)$	tuple
	$\mid$	$[a, \ldots, a]$	array
	$\mid$	a `++` a	concat
	$\mid$	a `at` x	localization
	$\mid$	a `:` τ	typing
p	$::=$		***Patterns***
	$\mid$	x	variable
	$\mid$	p `as` x	naming
	$\mid$	$d(p)$	descriptor

Static verification

Typing

$\Gamma \vdash e : \tau$

Memory location

$\Gamma \vdash e \mathbin{@} \varepsilon.f$

Static verification

Typing

$\Gamma \vdash e : \tau$

Memory location

$\Gamma \vdash e \mathbin{@} \varepsilon.f$

$$\frac{\Gamma \vdash e : \tau}{\Gamma \vdash !e : \tau} \quad (\textsc{Cpy})$$

Static verification

Typing

$\Gamma \vdash e : \tau$

$$\frac{\Gamma \vdash e : \tau}{\Gamma \vdash !e : \tau} \quad (\textsc{Cpy})$$

Memory location

$\Gamma \vdash e \mathbin{@} \varepsilon.f$

$$\frac{\Gamma \vdash e \mathbin{@} \varepsilon.f \qquad \varepsilon.f \cap \varepsilon'.f' = \emptyset}{\Gamma \vdash !e \mathbin{@} \varepsilon'.f'}$$

Static verification

Typing

$\Gamma \vdash e : \tau$

$$\frac{\Gamma \vdash e : \tau}{\Gamma \vdash !e : \tau} \quad (\textsc{Cpy})$$

$$\frac{\Gamma \vdash A : [n]\tau \qquad \Gamma \vdash B : [p]\tau}{\Gamma \vdash A{+}{+}B : [n+p]\tau} \quad (\textsc{Concat})$$

Memory location

$\Gamma \vdash e \mathbin{@} \varepsilon.f$

$$\frac{\Gamma \vdash e \mathbin{@} \varepsilon.f \qquad \varepsilon.f \cap \varepsilon'.f' = \emptyset}{\Gamma \vdash !e \mathbin{@} \varepsilon'.f'}$$

Static verification

Typing

$\Gamma \vdash e \ : \ \tau$

$$\frac{\Gamma \vdash e \ : \ \tau}{\Gamma \vdash \texttt{!}e \ : \ \tau} \quad (\textsc{Cpy})$$

$$\frac{\begin{array}{c}\Gamma \vdash A \ : \ [n]\tau \\ \Gamma \vdash B \ : \ [p]\tau\end{array}}{\Gamma \vdash A\texttt{++}B \ : \ [n+p]\tau} \quad (\textsc{Concat})$$

Memory location

$\Gamma \vdash e \ @ \ \varepsilon.f$

$$\frac{\Gamma \vdash e \ @ \ \varepsilon.f \qquad \varepsilon.f \cap \varepsilon'.f' = \emptyset}{\Gamma \vdash \texttt{!}e \ @ \ \varepsilon'.f'}$$

$$\frac{\begin{array}{l}\Gamma \vdash A \ @ \ \varepsilon.f \circ \left[\texttt{[i+0<}n+p\texttt{]} \ \leftarrow \ \texttt{.}1/2 \ \texttt{[i<}n\texttt{]}\right] \\ \Gamma \vdash B \ @ \ \varepsilon.f \circ \left[\texttt{[i+n<}n+p\texttt{]} \ \leftarrow \ \texttt{.}2/2 \ \texttt{[i<}p\texttt{]}\right]\end{array}}{\Gamma \vdash A\texttt{++}B \ @ \ \varepsilon.f}$$

Static verification

Typing

$\Gamma \vdash e \ : \ \tau$

$$\frac{\Gamma \vdash e \ : \ \tau}{\Gamma \vdash \texttt{!}e \ : \ \tau} \quad (\textsc{Cpy})$$

$$\frac{\begin{array}{c}\Gamma \vdash A \ : \ [n]\tau \\ \Gamma \vdash B \ : \ [p]\tau\end{array}}{\Gamma \vdash A\texttt{++}B \ : \ [n+p]\tau} \quad (\textsc{Concat})$$

Memory location

$\Gamma \vdash e \ @ \ \varepsilon.f$

$$\frac{\Gamma \vdash e \ @ \ \varepsilon.f \qquad \varepsilon.f \cap \varepsilon'.f' = \emptyset}{\Gamma \vdash \texttt{!}e \ @ \ \varepsilon'.f'}$$

$$\frac{\begin{array}{l}\Gamma \vdash A \ @ \ \varepsilon.f \circ \left[\texttt{[i+0<}n+p\texttt{]} \ \leftarrow \ \texttt{.1}/2 \ \texttt{[i<}n\texttt{]}\right] \\ \Gamma \vdash B \ @ \ \varepsilon.f \circ \left[\texttt{[i+n<}n+p\texttt{]} \ \leftarrow \ \texttt{.2}/2 \ \texttt{[i<}p\texttt{]}\right]\end{array}}{\Gamma \vdash A\texttt{++}B \ @ \ \varepsilon.f}$$

- No principal memory locations
- Semantics is independent of memory locations
- Locations filters out the set valid schedules

Modularity

Interfaces $\forall \varepsilon_1 : [n + p]\tau,\ \varepsilon_2 : [p]\tau,\ (\varepsilon_1.[],\ \varepsilon_2.[]) \to (\varepsilon_1.[]) \mid \varepsilon_1.[]$

```
let set_right (a, x) = (b at a):
  l ++ _ = a;
  b = l ++ !x;
```

Modularity

Interfaces $\boxed{\forall \varepsilon_1 : [n+p]\tau,\ \varepsilon_2 : [p]\tau}$, $(\varepsilon_1.[],\ \varepsilon_2.[]) \to (\varepsilon_1.[]) \mid \varepsilon_1.[]$

- Set of memory (typed) variables

```
let set_right (a, x) = (b at a):
  l ++ _ = a;
  b = l ++ !x;
```

Modularity

Interfaces $\forall \varepsilon_1 : [n+p]\tau,\ \varepsilon_2 : [p]\tau,\ \boxed{(\varepsilon_1.[],\ \varepsilon_2.[])} \to (\varepsilon_1.[]) \mid \varepsilon_1.[]$

- Set of memory (typed) variables
- List of input memory locations

```
let set_right (a, x) = (b at a):
  l ++ _ = a;
  b = l ++ !x;
```

Modularity

Interfaces $\forall \varepsilon_1 : [n+p]\tau,\ \varepsilon_2 : [p]\tau,\ (\varepsilon_1.[],\ \varepsilon_2.[]) \to \boxed{(\varepsilon_1.[])} \mid \varepsilon_1.[]$

- Set of memory (typed) variables
- List of input memory locations
- List of output memory locations

```
let set_right (a, x) = (b at a):
  l ++ _ = a;
  b = l ++ !x;
```

Modularity

Interfaces $\forall \varepsilon_1 : [n+p]\tau,\ \varepsilon_2 : [p]\tau,\ (\varepsilon_1.[],\ \varepsilon_2.[]) \to (\varepsilon_1.[]) \mid \boxed{\varepsilon_1.[]}$

- Set of memory (typed) variables
- List of input memory locations
- List of output memory locations
- List of edited memory locations

```
let set_right (a, x) = (b at a):
  l ++ _ = a;
  b = l ++ !x;
```

Modularity

Interfaces $\forall \varepsilon_1 : [n+p]\tau,\ \varepsilon_2 : [p]\tau,\ (\varepsilon_1.[],\ \varepsilon_2.[]) \to (\varepsilon_1.[]) \mid \varepsilon_1.[]$

- Set of memory (typed) variables
- List of input memory locations
- List of output memory locations
- List of edited memory locations

```
let set_right (a, x) = (b at a):
  l ++ _ = a;
  b = l ++ !x;
```

Instances

- Single-piece memory location for variables (Localization)

Modularity

Interfaces $\forall \varepsilon_1 : [n+p]\tau,\ \varepsilon_2 : [p]\tau,\ (\varepsilon_1.[],\ \varepsilon_2.[]) \to (\varepsilon_1.[]) \mid \varepsilon_1.[]$

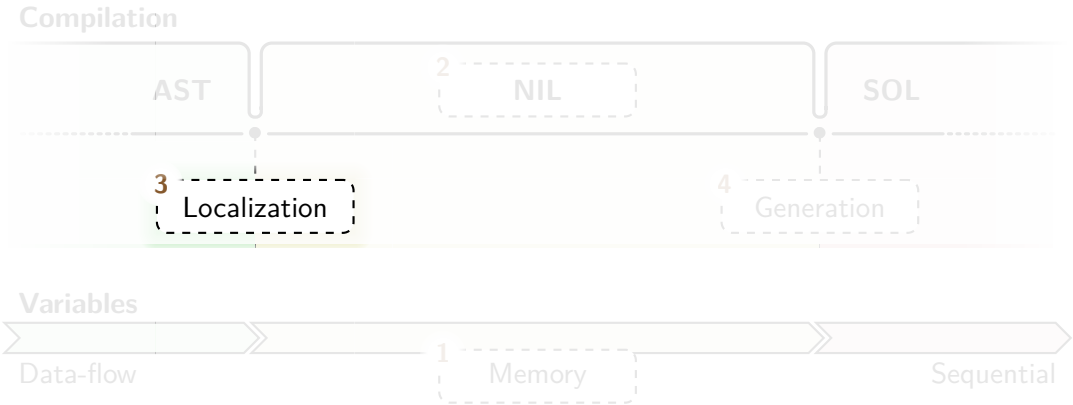
- Set of memory (typed) variables
- List of input memory locations
- List of output memory locations
- List of edited memory locations

```
let set_right (a, x) = (b at a):
  l ++ _ = a;
  b = l ++ !x;
```

Instances

- Single-piece memory location for variables (Localization)
- No alias for modified locations (Scheduling)

Agenda


Compilation
AST
2
NIL
SOL
3
Localization
4
Generation
Variables
Data-flow
1
Memory
Sequential

Location inference

Projection function unification: very under-constrained problem

- Restricted to simple unification (not inferring projection functions)
- Arrays and tuples build *single-piece* data

Location inference

Projection function unification: very under-constrained problem

- Restricted to simple unification (not inferring projection functions)
- Arrays and tuples build *single-piece* data

Errors

- ✗ Inconsistent locations
- ✗ Overlapping copies
- ✗ Undeclared sharing in interface (recursion)
- ✗ Non single-piece arguments (instantiation)

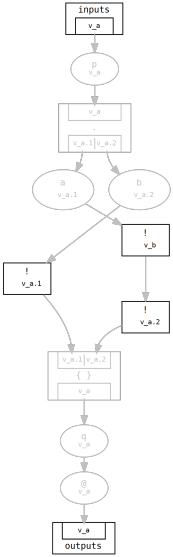
Dependencies analysis & scheduling

```
let swap (p) = (q at p)
    (a,b) = p;
    q = (!b, !!a);
```

Dependencies analysis & scheduling

```
let swap (p) = (q at p)
    (a,b) = p;
    q = (!b, !!a);
```

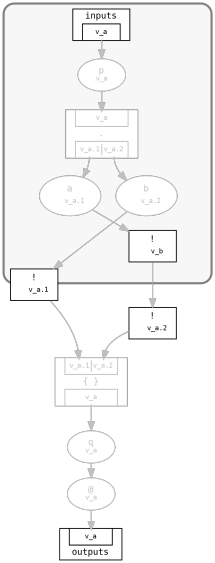
1. Data-flow graph extraction
   1.1 Data-flow construction

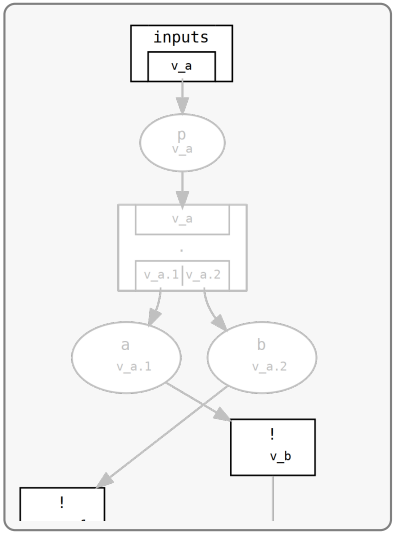


Dependencies analysis & scheduling

```
let swap (p) = (q at p)
    (a,b) = p;
    q = (!b, !!a);
```

1. Data-flow graph extraction
   1.1 Data-flow construction

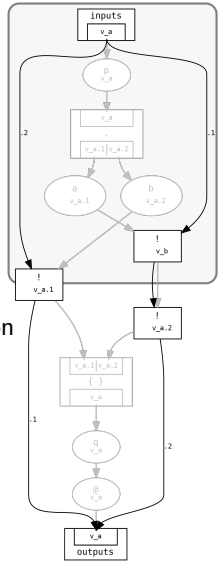


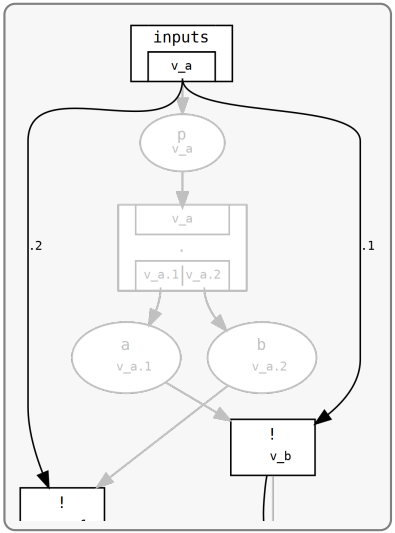


Dependencies analysis & scheduling

```
let swap (p) = (q at p)
    (a,b) = p;
    q = (!b, !!a);
```

1. Data-flow graph extraction
   1.1 Data-flow construction
   1.2 Data dependency computation

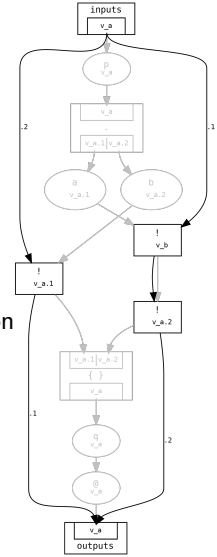


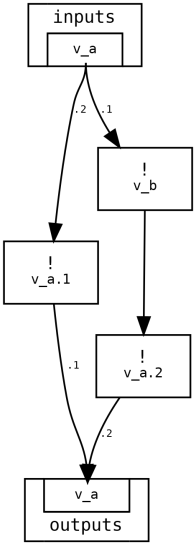


Dependencies analysis & scheduling

```
let swap (p) = (q at p)
    (a,b) = p;
    q = (!b, !!a);
```

1. Data-flow graph extraction
   1.1 Data-flow construction
   1.2 Data dependency computation
   1.3 Graph extraction

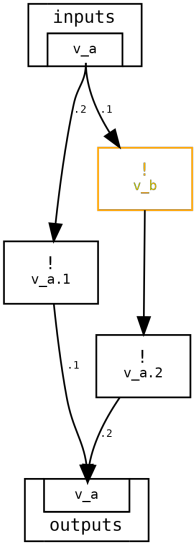




Dependencies analysis & scheduling

```
let swap (p) = (q at p)
    (a,b) = p;
    q = (!b, !!a);
```

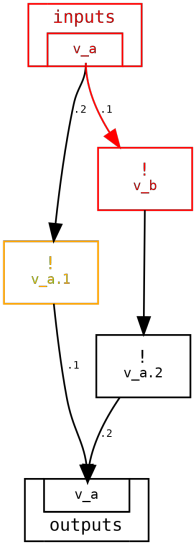
1. Data-flow graph extraction
   1.1 Data-flow construction
   1.2 Data dependency computation
   1.3 Graph extraction
2. Scheduling
   2.1 Interference analysis



Dependencies analysis & scheduling

```
let swap (p) = (q at p)
    (a,b) = p;
    q = (!b, !!a);
```

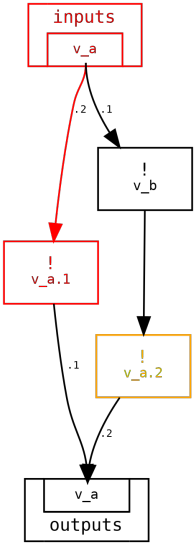
1. Data-flow graph extraction
   1.1 Data-flow construction
   1.2 Data dependency computation
   1.3 Graph extraction
2. Scheduling
   2.1 Interference analysis



Dependencies analysis & scheduling

```
let swap (p) = (q at p)
    (a,b) = p;
    q = (!b, !!a);
```

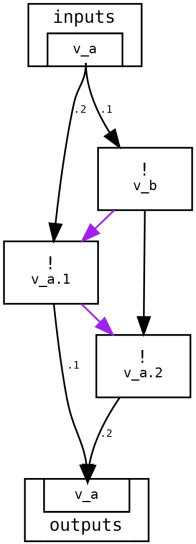
1. Data-flow graph extraction
   1.1 Data-flow construction
   1.2 Data dependency computation
   1.3 Graph extraction
2. Scheduling
   2.1 Interference analysis



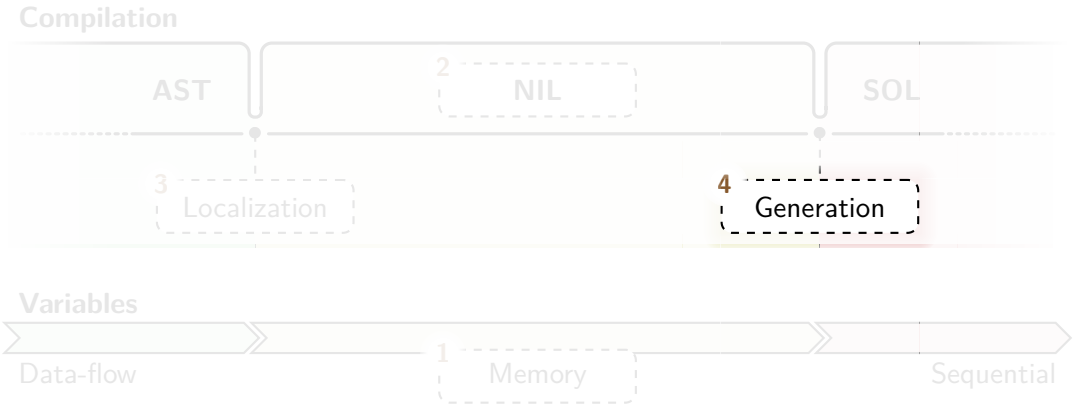
Dependencies analysis & scheduling

```
let swap (p) = (q at p)
    (a,b) = p;
    q = (!b, !!a);
```

1. Data-flow graph extraction
   1.1 Data-flow construction
   1.2 Data dependency computation
   1.3 Graph extraction
2. Scheduling
   2.1 Interference analysis
   2.2 Anti dependencies extraction



Agenda


Compilation
AST
2
NIL
SOL
3
Localization
4
Generation
Variables
Data-flow
1
Memory
Sequential

C representation of data

Data representation

- Raw data for memory variables (no indirections)
- Pointer arithmetic for slices / accesses

C representation of data

Data representation

- Raw data for memory variables (no indirections)
- Pointer arithmetic for slices / accesses

Copy implementation: concretizing projection functions

- Simple assignment for scalar values
- `memcpy` for single-piece locations
- Multiple statements (loops) for other cases.

The `step` function

Calling conventions:

- Optional state (pointer)
- Pointers on memory variables by default
- Except for unmodified scalar inputs
- Except for one scalar output (fresh)

The `step` function

Calling conventions:

- Optional state (pointer)
- Pointers on memory variables by default
- Except for unmodified scalar inputs
- Except for one scalar output (fresh)

$$\texttt{val ex:}\ \forall \left\{ \begin{array}{ll} \varepsilon_1 : \texttt{int} & (\mathrm{i}) \\ \varepsilon_2 : \texttt{int} * \texttt{int} & (\mathrm{p}) \\ \varepsilon_3 : \texttt{bool} & (\mathrm{b}) \end{array} \right\}, \quad \frac{(\varepsilon_1.[],\ \varepsilon_2.[]) \rightarrow (\varepsilon_2.\left[.1/2 \leftarrow\right],\ \varepsilon_3.[])}{\varepsilon_2.[],\ \varepsilon_3.[]}$$

The step function

Calling conventions:

- Optional state (pointer)
- Pointers on memory variables by default
- Except for unmodified scalar inputs
- Except for one scalar output (fresh)

$$\texttt{val ex:}\ \forall \left\{ \begin{array}{ll} \varepsilon_1 : \texttt{int} & (\texttt{i}) \\ \varepsilon_2 : \texttt{int} * \texttt{int} & (\texttt{p}) \\ \varepsilon_3 : \texttt{bool} & (\texttt{b}) \end{array} \right\}, \quad \frac{(\varepsilon_1.[],\ \varepsilon_2.[]) \rightarrow (\varepsilon_2.\left[.1/2 \leftarrow\right],\ \varepsilon_3.[])}{\varepsilon_2.[],\ \varepsilon_3.[]}$$

```
void ex_step (ex_state *_st);
```

The step function

Calling conventions:

- Optional state (pointer)
- Pointers on memory variables by default
- Except for unmodified scalar inputs
- Except for one scalar output (fresh)

$$\texttt{val ex:}\ \forall \left\{ \begin{array}{ll} \varepsilon_1 : \texttt{int} & (\texttt{i}) \\ \varepsilon_2 : \texttt{int} * \texttt{int} & (\texttt{p}) \\ \varepsilon_3 : \texttt{bool} & (\texttt{b}) \end{array} \right\}, \quad \frac{\begin{array}{l} (\varepsilon_1.[\,],\ \varepsilon_2.[\,]) \\ \to (\varepsilon_2.\left[.1/2\ \leftarrow\right],\ \varepsilon_3.[\,]) \end{array}}{\varepsilon_2.[\,],\ \varepsilon_3.[\,]}$$

```
void ex_step (ex_state *_st, int i);
```

The step function

Calling conventions:

- Optional state (pointer)
- Pointers on memory variables by default
- Except for unmodified scalar inputs
- Except for one scalar output (fresh)

$$\texttt{val ex:}\ \forall \left\{ \begin{array}{ll} \varepsilon_1 : \texttt{int} & (\texttt{i}) \\ \varepsilon_2 : \texttt{int} * \texttt{int} & (\texttt{p}) \\ \varepsilon_3 : \texttt{bool} & (\texttt{b}) \end{array} \right\}, \quad \frac{(\varepsilon_1.[],\ \boxed{\varepsilon_2.[]}) \to (\varepsilon_2.\left[.1/2 \leftarrow\right],\ \varepsilon_3.[])}{\boxed{\varepsilon_2.[]},\ \varepsilon_3.[]}$$

```
void ex_step (ex_state *_st, int i, tpl_ii *p);
```

The step function

Calling conventions:

- Optional state (pointer)
- Pointers on memory variables by default
- Except for unmodified scalar inputs
- Except for one scalar output (fresh)

$$\texttt{val ex:}\ \forall \left\{ \begin{array}{lr} \varepsilon_1 : \texttt{int} & (\texttt{i}) \\ \varepsilon_2 : \texttt{int} * \texttt{int} & (\texttt{p}) \\ \varepsilon_3 : \texttt{bool} & (\texttt{b}) \end{array} \right\}, \quad \frac{(\varepsilon_1.[],\ \varepsilon_2.[]) \to (\varepsilon_2.[.1/2 \leftarrow],\ \varepsilon_3.[])}{\varepsilon_2.[],\ \varepsilon_3.[]}$$

```
void ex_step (ex_state *_st, int i, tpl_ii *p);
```

The step function

Calling conventions:

- Optional state (pointer)
- Pointers on memory variables by default
- Except for unmodified scalar inputs
- Except for one scalar output (fresh)

$$\texttt{val ex:}\ \forall \left\{ \begin{array}{ll} \varepsilon_1 : \texttt{int} & (\texttt{i}) \\ \varepsilon_2 : \texttt{int} * \texttt{int} & (\texttt{p}) \\ \varepsilon_3 : \texttt{bool} & (\texttt{b}) \end{array} \right\}, \quad \frac{(\varepsilon_1.[],\ \varepsilon_2.[]) \rightarrow (\varepsilon_2.[.1/2 \leftarrow],\ \boxed{\varepsilon_3.[]})}{\varepsilon_2.[],\ \boxed{\varepsilon_3.[]}}$$

```
bool ex_step (ex_state *_st, int i, tpl_ii *p);
```

The step function

Calling conventions:

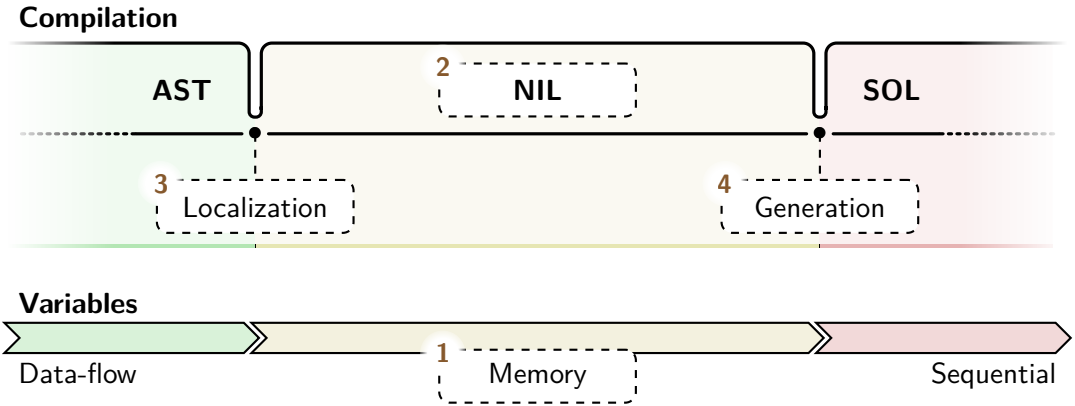
- Optional state (pointer)
- Pointers on memory variables by default
- Except for unmodified scalar inputs
- Except for one scalar output (fresh)

$$\texttt{val ex:}\ \forall \left\{ \begin{array}{ll} \varepsilon_1 : \texttt{int} & (\texttt{i}) \\ \varepsilon_2 : \texttt{int} * \texttt{int} & (\texttt{p}) \\ \varepsilon_3 : \texttt{bool} & (\texttt{b}) \end{array} \right\}, \quad \frac{(\varepsilon_1.[],\ \varepsilon_2.[]) \\ \to (\varepsilon_2.\left[.1/2\ \leftarrow\right],\ \varepsilon_3.[])}{\varepsilon_2.[],\ \varepsilon_3.[]}$$

```
bool ex_step (ex_state *_st, int i, tpl_ii *p);
```

Demo

Agenda


Compilation
AST
2
NIL
SOL
3
Localization
4
Generation
Variables
Data-flow
1
Memory
Sequential

What about SCADE?

Language: a conservative extension

- Implicit copies: interface, data-construction
- Copy-less assignments and projections (`x = y.1;`)
- Copy-less operators (array in-place update)
- Array combinators for index functions (concatenation, transposition, ...)
- Explicit copy operator

What about SCADE?

Language: a conservative extension

- Implicit copies: interface, data-construction
- Copy-less assignments and projections (`x = y.1;`)
- Copy-less operators (array in-place update)
- Array combinators for index functions (concatenation, transposition, ...)
- Explicit copy operator

Extended causality check: semi-linearity[2]

- Location may be modified at most once
- Simpler & more precise diagnostics
- Sufficient condition for scheduling

[2] Léonard Gérard et al. "A Modular Memory Optimization for Synchronous Data-Flow Languages. Application to Arrays in a Lustre Compiler". In: *Languages, Compilers and Tools for Embedded Systems (LCTES'12)*. Best paper award. ACM. Beijing, 2012. URL: `lctes12.pdf`.

Take away

A fully parallel imperative language

- Data-flow representation in a pure SSA form

Take away

A fully parallel imperative language

- Data-flow representation in a pure SSA form
- Memory specification with semantic preserving annotations

Take away

A fully parallel imperative language

- Data-flow representation in a pure SSA form
- Memory specification with semantic preserving annotations
- Make all copies explicit

Take away

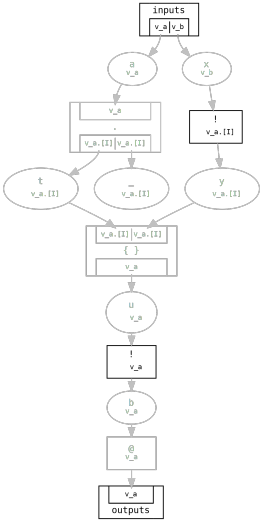
A fully parallel imperative language

- Data-flow representation in a pure SSA form
- Memory specification with semantic preserving annotations
- Make all copies explicit
- Referential transparency

Thank you!

Will linear types change the world?[3]

```
let set_right (a,x) = (b at a)
  y = !x;
  t ++ _ = a;
  u = t ++ y;
  b = !u;
```

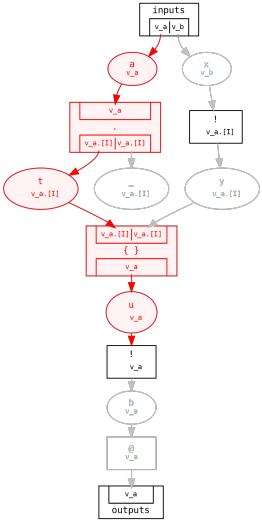


[3] Philip Wadler. "Linear types can change the world!" In: *Programming concepts and methods*. Vol. 3. 4. Citeseer. 1990, p. 5.

Will linear types change the world?[3]

```
let set_right (a,x) = (b at a)
  y = !x;
  t ++ _ = a;
  u = t ++ y;
  b = !u;
```

	Linearity	Interferences
In place update	✓	✓

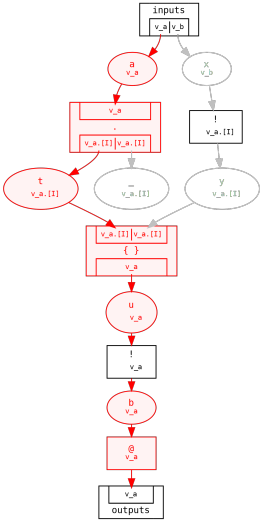


[3] Philip Wadler. "Linear types can change the world!" In: *Programming concepts and methods*. Vol. 3. 4. Citeseer. 1990, p. 5.

Will linear types change the world?[3]

```
let set_right (a,x) = (b at a)
  y = !x;
  t ++ _ = a;
  u = t ++ y;
  b = !u;
```

	Linearity	Interferences
In place update	✓	✓
Interface/ Register	(✓)	✓

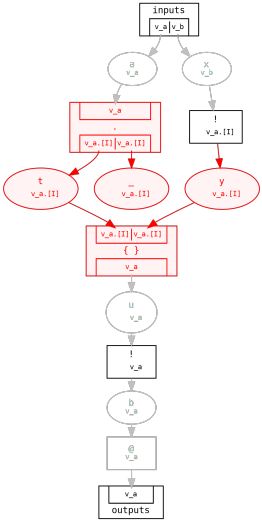


[3] Philip Wadler. "Linear types can change the world!" In: *Programming concepts and methods*. Vol. 3. 4. Citeseer. 1990, p. 5.

Will linear types change the world?[3]

```
let set_right (a,x) = (b at a)
  y = !x;
  t ++ _ = a;
  u = t ++ y;
  b = !u;
```

	Linearity	Interferences
In place update	✓	✓
Interface/ Register	(✓)	✓
Built-in place	✗	✓

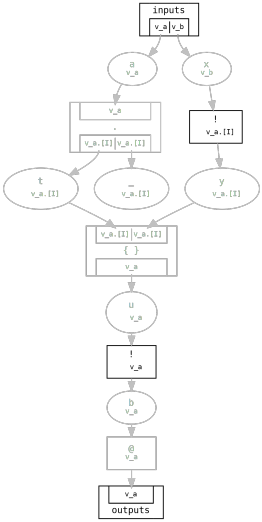


[3] Philip Wadler. "Linear types can change the world!" In: *Programming concepts and methods*. Vol. 3. 4. Citeseer. 1990, p. 5.

Will linear types change the world?[3]

```
let set_right (a,x) = (b at a)
  y = !x;
  t ++ _ = a;
  u = t ++ y;
  b = !u;
```

	Linearity	Interferences
In place update	✓	✓
Interface/ Register	(✓)	✓
Built-in place	✗	✓
Error reporting	✓	(✓)



[3] Philip Wadler. "Linear types can change the world!" In: *Programming concepts and methods*. Vol. 3. 4. Citeseer. 1990, p. 5.